CSC A48 – Introduction to Computer Science II

Hello and welcome to A48! Please take a moment to read this handout carefully, it will give you an overview of the course, what you can expect to learn from it, the skills you will develop, and the general course policies.

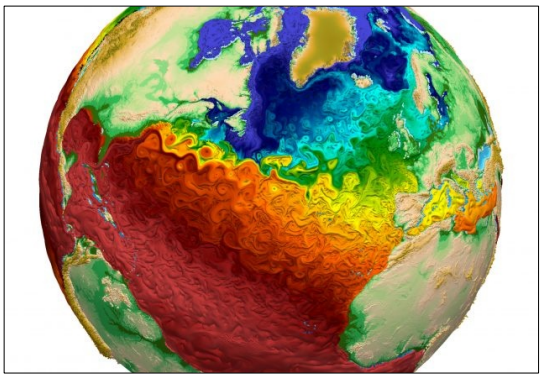
1.- Introduction

This course is the place where we build the foundations of knowledge you will need for any future task that requires using a computer to solve a problem. The course is all about ***general techniques*** used to design and implement programs for a wide range of applications and domains. As such, having a solid grasp of what you will learn here will enable you to apply computer science techniques to your own field of work:

If you are in Computer Science – this course will prepare you for the core of your career. Every course from second year onward assumes you can apply the techniques developed in this course as a framework on which to build more advanced, topic-specific concepts. A48 is directly relevant for *Software Design* (B07), *Intro to the Theory of Computation* (B36), *Software Tools and System Programming* (B09), *Analysis of Algorithms and Data Structures* (B63), and even *Computer Organization* (B58). ***The work you put into this course will translate into making the most of all those courses.***

If you are in Math, or Stats – this course may be even more important to you. Unlike CS specialists/ majors, you don't have the luxury of multiple courses over which to develop the ability to create a computer-based solution for a given task. Therefore it is ***essential*** that you make the most of A48. You will use what you learn here for applications in ***financial technologies,*** and ***data analytics***. You will also find that knowing about computing, and being able to think about a problem from a computer science perspective will create opportunities for interesting work in the future.

What we will study this term



Climate modelling. One of many critically important problems that computer science can help us tackle.
Image: Los Alamos National Labs (Public Domain)

- What information do we need to store for a given application? How do we form a collection of data that is searchable efficiently?

- What is *efficiency* in computational terms? Given different ways to solve a problem, how do we choose the best one?

- What is the *fastest* way to search through large amounts of data for something we want?

- What is *recursion*, and what kinds of problems can we solve with it?

- How do we organize software to make it re-usable, easy to maintain, test, and build upon?

2.- Course Learning Goals

With hard work and practice, by the end of this course you will be able to:

- Design and implement custom data containers to store information for any application you need to work with. This includes figuring out what data needs to be stored, and how to keep a large number of data items properly organized so the data is usable, can be searched and updated, and the chosen organization is suitable for the task you're interested in.

- Think in concrete terms about the amount of work that will be required to perform common data manipulation operations, such as searching, updating, adding new data items, or deleting items no longer needed. This will allow you to choose among different possible ways to organize and store information the one that is most suitable for a given problem.

- Implement and use a couple of the most common data structures (these are ways to structure, organize, and manage information), and identify their advantages and disadvantages so you can apply the optimal one to a given problem.

- Identify problems that can be solved in a *recursive* way, formulate and implement a recursive solution for these problems.

- Write software in a way that makes it re-usable, easy to maintain and build-upon, and that enables others to easily incorporate your code into their projects, modify it, and expand it as needed.

3.- Skills that you will develop (once again, provided you work hard and practice!)

- Identifying which of the techniques covered in class can be applied toward solving a particular problem.

- Understanding what happens with data in memory as a program runs. Being able to explain why data changed the way it did, and to identify and fix bugs related to how information is passed around in a program.

- Writing, testing, and debugging programs that implement the methods discussed in class.

- Thinking in detail about a program, understanding every single thing that happens in it, and why it has to happen that way (conversely, being able to tell when it's doing the wrong thing and why!)

- Good work and study habits that will allow you to *learn in-depth* and make the most of any technical courses you will take through your career.

- Making use of all the resources available to improve your ability to learn: Access to course instructors, TAs, exercises, problem sets, assignments, and reference material.

- Building a good environment for yourself and your peers so the next few years are awesome!

4.- Practical information to keep in mind

Our programming language this term will be ***C***. Like its name, it is short and simple, and perfectly suited for exercising your ability to understand what is going on inside a program.

All the lab computers at UTSC have the required software and editor so you can do your course work, however, if you wish to set up your laptop or home computer, you need to download and install:

- The GNU project compiler suite ***gcc*** (any recent version will do)
- A suitable *text editor* with *syntax highlighting*. On Windows, try ***notepad++***
  (avoid using an IDE until you have learned to work from the command line!***)***

Instructions on how to do this for Windows can be found here:

https://www.scaler.com/topics/c/c-compiler-for-windows/

On Linux, simply open your distribution's software manager, search for ***gcc*** and install the compiler and documentation. You can use ***kate*** as a text editor for programming, so go ahead and install that as well.

If you're using Mac, you will be using the ***clang*** compiler, it will work just fine, all you need to is replace 'gcc' with 'clang' in all the instructions we give you to compile code:

https://www.ics.uci.edu/~pattis/common/handouts/macclion/clang.html

Forum:

We will have a class forum on ***Piazza***. All students will be enrolled at the start of term with the email addresses we have on record. *It is your responsibility to check that email address to get access to the forum.*

Ground rules for using the forum effectively:

The purpose of the forum is to provide a place for students to discuss material and help each other with *assistance and supervision* from the TAs and course instructors. ***It is not a substitute for office hours, tutorials, labs, or lectures.*** It is unreasonable to expect the TAs or instructors to continually hover over the forum just so that questions can be answered within minutes. ***Do not expect this.***

Do not post solutions to the forum – this includes exercises, problem sets, and especially assignments. Helping each other is good, giving away solutions is not.

Everyone in the forum is expected to behave professionally and with courtesy. The forum is not a place for venting frustration. It is a professional environment and everyone will treat it in that way. Anyone caught behaving unprofessionally will have their access revoked.

4.- Practical information to keep in mind (cont.)

Ground rules for using the forum effectively…

Things to do at the forum:

- Seek clarification about specific things from lecture, the notes, or coursework.

- Seek help with technical problems (e.g. dealing with obscure compiler messages)

- Help our peers with material you have learned (best way to make sure you learned)

- Keep the discussion ***on topic***. If you want to ask a question about anything that is beyond what the course covers, *please tag the question as* ***out-of-scope***. We'll be happy to look at it, and we want you to enjoy the course and learn as much as you can, but everyone needs to know that this question is not part of what they need to master to do well in the course.

Before you post:

- ***Try hard*** to figure out the answer to your question before you post it – review the notes, work through related examples, and figure out what exactly is giving you trouble. Often this is enough to help you figure out the solution, and you will learn more from it.

- Think about whether what you're going to post could be said during lecture, in front of everyone, with a microphone in your hand. The forum is as public a space as any other.

- Consider visiting your instructor or one of the TAs during office hours: You will have the dedicated attention of a person who is experienced in the material and happy to help!

Your weekly work plan

The course has been very carefully organized so everything is in place to help you learn, your job is to do the work that will result in you mastering the course material. Every week you have to:

- ***Carefully go through the assigned readings.*** The lecture notes are very detailed and cover every important aspect of what you need to learn. *You have to do the readings in advance, before your lecture.* In lecture, we will *go over the essential ideas* from the lecture, *make sure those are clear*, and *solve problems*. ***If you don't do the readings, you won't make the most of the lecture.***

- ***Attend your tutorial.*** Tutorials will be used to do collaborative problem-solving, and will also cover technical aspects of programming in ***C*** that are not covered in lecture. Do not skip them!

4.- Practical information to keep in mind (cont.)

Your weekly work plan…

- ***Practice!*** You will have to complete ***weekly exercises*** (these will be submitted, but ***not marked***) ***However*** they are part of your final mark – you get points simply for working on the exercises and submitting your completed work.

- ***Visit the office hours***. Get help when you need it, there will be plenty of office hours between your TAs and the course instructors. ***These do nothing for you unless you actually go!***

- ***End of week questionnaires.*** Every week, you will fill-in a little questionnaire on Quercus. It will help us figure out how the course is going, and where people need help. It will help you reflect on what you learned each week, and where you may need to focus your effort. Completing these questionnaires contributes a small part of your final mark. ***But they are not marked***.

5.- Resources

Your best resource this term are the ***course notes*** that will be distributed at the start of each week. You will be tested on concepts, ideas, and problems that appear throughout those notes. ***You do not need a textbook for this course.***

Any aspects of C programming that you need to master to get through the course are covered in the notes, or will be covered in tutorial.

However, it is always good to have a few extra resources when dealing with a new programming language. Here's a couple of links to good tutorial sites (feel free to find and use your own though):

https://www.programiz.com/c-programming#tutorial

https://www.learn-c.org/

We will make more links and resources available on the Quercus site for the course once the term Begins.

6.- Course administration

Course instructors: Dr. Marcelo Ponce, Dr. Paco Estrada
Course coordinator: Paco Estrada, Office at IC494

Office hours: To be posted on Quercus – these are open to all students in A48, not limited to who is the instructor for your lecture section! Talk with everyone, and find out whose style best helps you learn.

TA office hours and lab schedule: To be posted on Quercus during week 1

Grading:

Item	Notes	Value
Weekly exercises (auto-tested)	Develop problem solving ability.	5% (only your best 9 count)
Weekly questionnaires (not marked)	Help you reflect on your learning	5%
Assignments (x3)	***Auto-tested***	15%
Tutorial Exercises	***Not marked***	5%
Midterm test	Date TBA	20%
Final exam	Date TBA	50%

Academic Integrity:

The course work is intended to ***help you master the concepts and ideas*** the course is all about. there is ***no point*** in getting a high mark in assignments that is not the product of your own hard work and the learning it brought you. So, ***you're expected to figure out and implement solutions to the assignments by yourself.*** Getting ***any component of the solution (including a verbal step-by-step explanation of how to do it) from another student, social-networking apps e.g. WeChat, or 'Mentoring' services, etc. is an academic offense***.
Read the following handout for tips on how to stay out of trouble:

https://www.utsc.utoronto.ca/aacc/sites/utsc.utoronto.ca.aacc/files/tipsheets/AIM%20-%20Tipsheet%20oct%202015.pdf

Keep in mind – if you do not solve your assignments on your own, you will not have learned what the assignment is intended to help you learn, and it will cost you on quizzes and exams.

Sessional dates:
https://www.utsc.utoronto.ca/registrar/fall-2021-winter-2022-academic-dates